

Final Report, A Guide to the Architecture Document

D7065E, what each section must contain, and the ideas behind it

Course staff

Contents

1. What the architecture document is	1
2. The ideas behind the document	1
2.1 Model-Based Systems Engineering, lightly	1
2.2 The C4 model	2
2.3 The five architecture viewpoints	3
2.4 Communication patterns	4
2.5 Deployment and component placement	4
2.6 How to justify a decision	5
3. Section-by-section guide	5
3.1 Summary	5
3.2 Use case and building context (BuildSim)	5
3.3 Requirements	5
3.4 Architecture	6
Context diagram (C4 Level 1)	6
Container diagram (C4 Level 2)	6
Component responsibilities and design decisions	8
3.5 Interfaces and communication	8
3.6 Simulating the sensor values	8
3.7 Autonomous services and data engineering	8
3.8 Behaviour	9
3.9 Deployment and component placement	10
3.10 Test plan	10
3.11 Evaluation and experiments	10
3.12 Dashboard, risks and reflection	10
4. Checklist before submission	10

How to use this guide. This document explains *how to write the architecture document* that forms the core of the final report: what each section is for, what it must contain, and the ideas (MBSE, C4, viewpoints) behind them. It is the companion to the worked report example, which shows a complete, filled-in version at the grade-5 standard. Read this guide to understand *why*; read the example to see *what good looks like*. The architecture document is a **living document**: start it once the Week 2 proposal is approved and refine it as the project is built, so the document and the code evolve together. The final version is submitted with the report and should match what was actually built.

1. What the architecture document is

The architecture document is the **contract between the design and the implementation**, and it is the main written evidence an examiner reads. It is where the three kinds of thinking the course is built to develop come together:

- **Architectural thinking**, modelling the system precisely before building it, and justifying every design choice against the alternatives that were rejected.
- **Systems thinking**, reasoning about the whole feedback loop, not isolated parts: how data flows, where computation belongs, and how a failure in one component ripples through the rest.
- **Critical thinking**, judging from evidence how well the system works, finding where it breaks, weighing trade-offs, and being honest about its weaknesses.

A small, well-architected, well-justified, critically-evaluated system scores higher than a large one that “just works” but cannot be explained or defended. The document is graded across four dimensions (A–D in the grading criteria); most of the architecture grade is decided by the quality of the reasoning, not by the number of features.

2. The ideas behind the document

Before the section-by-section guidance, it is worth understanding the ideas the whole document rests on. Getting these straight is what separates a report that lists technologies from one that demonstrates understanding.

2.1 Model-Based Systems Engineering, lightly

The course is grounded in **Model-Based Systems Engineering (MBSE)**, but it is easy to mistake the notation for the point, so be clear about what it means here.

The substance of MBSE is architectural and systems thinking, reasoning about requirements, components, interfaces, behaviour, and the feedback loop *before* code is written, so design errors surface while they are still cheap to fix. Errors found during design cost minutes; the same errors found after deployment cost days, or, in a building-control system, worse. That early reasoning is the goal, and it is what this document captures.

SysML and heavyweight MBSE tooling are just one way to write that thinking down, and this course does not use them. Instead the course uses a few precise, lightweight artefacts, C4 diagrams drawn as code (e.g. with D2), a requirements table, interface contracts, and a couple of behavioural diagrams, kept in the repository next to the code. Using lightweight artefacts is *not* a departure from MBSE: it keeps MBSE’s core (the thinking) and sheds the notation overhead. What is assessed is the quality of the architectural, systems, and critical reasoning and the ability to justify the decisions, not fluency with a modelling language.

The single most important MBSE habit is **traceability**: every requirement should connect to a design element that satisfies it and a test that verifies it. If a requirement owns no component, the design has a gap; if it has no test, the requirement cannot be enforced. This “requirement → design → validation” thread runs from the requirements table through the architecture sections to the test plan, and it is the backbone of a strong document.

2.2 The C4 model

C4 is a lightweight way to draw software architecture, created by Simon Brown specifically because UML felt too heavy for most teams. The name comes from its **four levels of zoom**, each adding

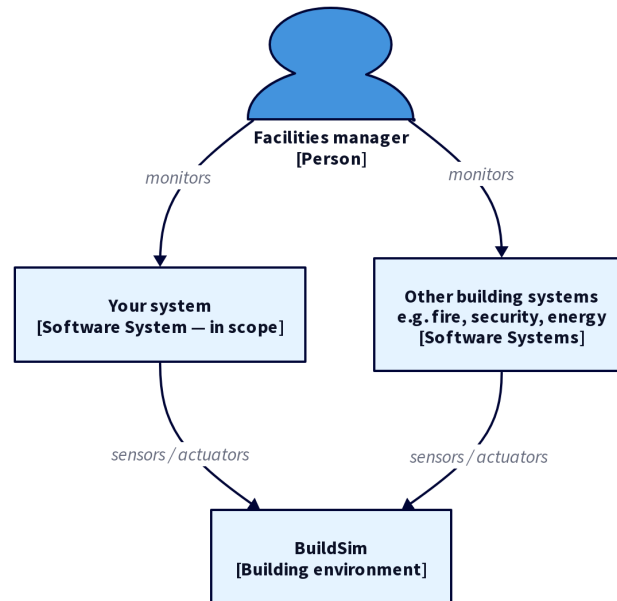
detail. The key idea is to draw the system at the level of detail the reader needs, never everything at once.

Level	Name	What it shows	Analogy (Google Maps)
1	Context	The system as a single box, surrounded by its users and external systems	The country on the globe
2	Container	The separately-deployable units inside (processes, containers, databases) and how they talk	The cities and roads
3	Component	The internal parts of <i>one</i> container	The streets within a city
4	Code	Class-level detail (rarely drawn by hand)	Individual buildings

For this course, **Levels 1 and 2 are what must be produced**, and they carry most of the architecture grade.

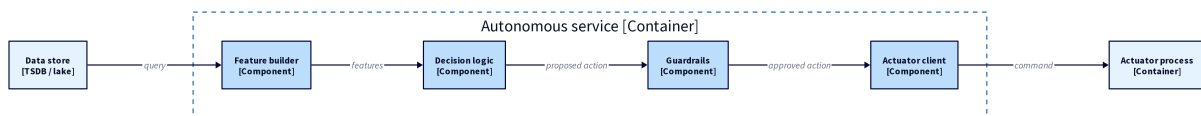
- The **Context diagram (Level 1)** fixes the *system boundary*: what is yours to build versus what is outside it (the building manager, occupants, BuildSim, optional external compute). Nothing internal is shown. Its job is to make the scope unambiguous.
- The **Container diagram (Level 2)** opens the box and shows every deployable unit, each box is roughly one Docker container, with the protocol on every arrow. This is the most important diagram in the document, because it should map **1:1 to the docker-compose.yml and the repository folders**. If the diagram shows six containers and the repository jams everything into one folder, the design and the code have parted ways.
- A **Component diagram (Level 3)** is optional, worth drawing only when one container is complex enough to deserve its own picture (for example an autonomous service with several internal modules).

Beyond the four core levels, C4 has **supplementary** diagrams that may be used: a **System Landscape** (the big picture, several systems and people in one view, no single system in focus), a **Dynamic** diagram (how elements collaborate at runtime for one scenario, with numbered interactions, this is a sequence-style view, in C4 terms), and a **Deployment** diagram (what runs where). **Do not draw Level 4 (Code) by hand**, C4 itself advises against it, since class-level diagrams drift from the code immediately and are better generated by an IDE if ever needed.



C4 System Landscape, several building systems, each on BuildSim, overseen by the facilities team; the in-scope system is highlighted.

A **Level 3 Component** diagram is worth drawing for the one container complex enough to need it (typically the autonomous service):



C4 Component diagram (Level 3), the internals of one container, its components in light blue.

C4 is technology-agnostic and focuses on structure and relationships, which is exactly the lightweight modelling MBSE asks for.

2.3 The five architecture viewpoints

No single diagram can serve every reader: a developer needs to see software components, an operator needs to see machines and networks, a safety reviewer needs to see failure modes. Trying to show all of that at once produces an unreadable tangle. The solution is **multiple viewpoints**, each a lens that filters out everything not relevant to one concern, and each catching a kind of error the others hide.

A complete architecture description uses five viewpoints, and they map onto the sections of the report:

Viewpoint	Question it answers	Where it lives in the report
Context	What interacts with the system?	C4 context diagram
Functional	What are the parts, and how do they connect?	C4 container diagram + responsibilities
Information	What data exists, and how does it flow?	Data models, interfaces, pipeline
Behavioural	What happens, in what order, for a scenario?	Sequence and state diagrams
Deployment	What runs where?	Deployment diagram + component placement

A correct functional view can still hide a broken deployment view (two components that look connected might need a network link that does not exist); a clean data model can hide a behavioural timing bug that only a sequence diagram exposes. Covering all five avoids leaving a class of error undiscovered until the demo.

2.4 Communication patterns

Most architectural mistakes at this scale come from choosing the wrong way for components to talk. Many patterns are not needed, the right one must be picked for each link and justify it. The three that matter here:

- **Request–response (REST over HTTP).** One component asks another a specific question and waits. Simple, tightly coupled, no buffering. Right for **commands and queries**, setting an actuator’s state and confirming it, registering a device, reading a value. Wrong for high-frequency streams and for one-to-many fan-out.
- **Publish–subscribe (MQTT, Kafka).** Publishers send to a *topic* without knowing who listens; a broker routes to subscribers. Loosely coupled, buffers when a consumer is briefly down. Right for **sensor data distribution**, where one sensor feeds many consumers (the database, the autonomous service, the dashboard).
- **WebSocket.** A persistent two-way connection so the server can push to the client without polling. Right for **live dashboards**; BuildSim already uses it to stream the 3D view.

The rule of thumb: **REST when one component asks another and waits; pub/sub when one source feeds many; WebSocket when the server pushes to a browser.** State which is used on each link, and why.

2.5 Deployment and component placement

In this course everything runs on the laptop, so deployment is less about *where in the world* a component runs and more about **how the system is composed from independent processes**. The deployment viewpoint answers: what runs as its own process, how those processes are isolated and packaged, how they communicate, and how the system behaves when one fails. The concerns that matter:

- **Process isolation and containerization.** Each component runs as its own containerized process, so it can be started, stopped, and restarted independently, and a fault in one does not take down the others.
- **Communication.** Processes talk only through defined interfaces (Section 2.4), never shared memory, which is what lets them be developed and deployed independently.
- **Resource allocation.** Components have different needs, storage for the pipeline, CPU for the autonomous service, little for the dashboard, and separate processes let each be provisioned independently.
- **Fault isolation.** If the dashboard crashes, the autonomous service keeps running. This is the operational payoff of one responsibility per process.

The **edge–cloud continuum** is still worth understanding as part of the syllabus: a real building would run the control loop on a local *edge* server so it keeps working when the internet is down, and use the *cloud* or a GPU server only for slow work off the critical path (such as overnight model training). In this course that whole continuum collapses onto one laptop, but the deployment section should still reason about which components would belong at the edge and which could be pushed to external compute, and why.

2.6 How to justify a decision

Most of the architecture grade comes from *justification*, and there is a simple pattern that turns an assertion into a decision:

“X was chosen because ... (tie it to a requirement); the alternative was Y; the trade-off accepted is Z.”

“MQTT was used” is an assertion. “MQTT was used rather than direct REST because three consumers need the same readings and the broker buffers them if one is briefly down (NFR-02); the trade-off is one more process to run and monitor” is a decision. A decision table with no rejected alternatives and no trade-offs reads as a list of assumptions, write the alternatives down even when the choice feels obvious.

3. Section-by-section guide

The rest of this guide walks through each section of the architecture document. Replace the **bracketed guidance** with real content; the illustrative tables in the worked example show the level of detail expected.

3.1 Summary

A short overview so a reader knows, in one glance, what the system is and where to find things: project title, team, use case, repository, and version. Include a brief **“changes since the proposal”** note, what is different from the Week 2 proposal and *why*. The examiner explicitly looks for this, because it shows the design evolved through reasoning rather than by accident.

3.2 Use case and building context (BuildSim)

Describe the **use case** and how it connects to **BuildSim**. BuildSim is the *shared state* of the cyber-physical system: it owns the building model (rooms, equipment), exposes a REST API to register devices and read/write their state, and streams the 3D view over WebSocket. BuildSim is not modified; it is instrumented. State explicitly how the processes use it, the simulator writes the sensor values into BuildSim and reads actuator state back (it is the only process that touches the environment), sensor processes read those values from BuildSim and publish them onward, actuator processes write their state to BuildSim, and the dashboard rides BuildSim’s WebSocket. Name the actors (building manager, occupants), tell the concrete scenario the demo will show, and explain why the use case matters. This section is the *context viewpoint* in prose.

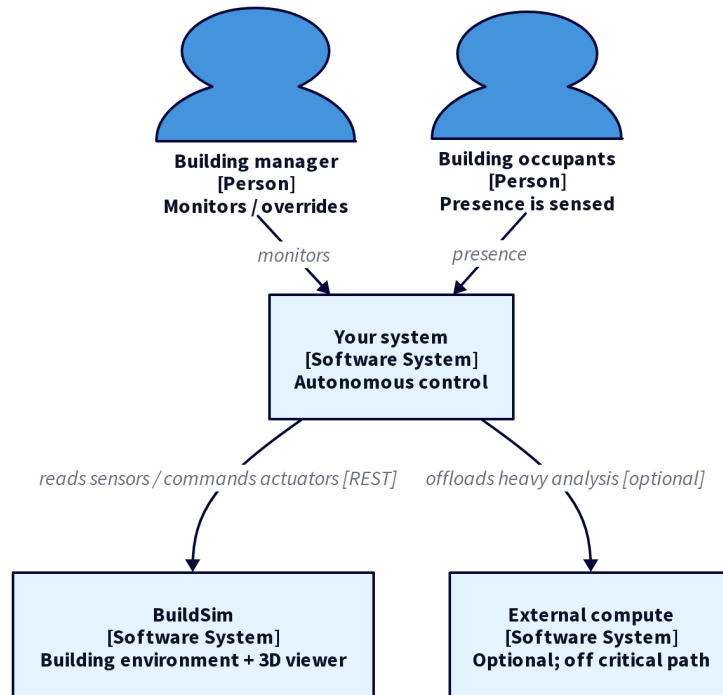
3.3 Requirements

State what the system must do as **testable** statements. Each requirement gets a unique **ID**, a **type**, Functional (what it does), Non-functional (how well: latency, reliability, recovery), or Regulatory (an external rule), a **priority** (Must / Should / Could), a **measurable acceptance criterion**, and the **test** that verifies it. Non-functional requirements matter most for grading: they are what the design decisions get justified against. Quantify everything, “the system should be fast” cannot be tested; “a decision every 60 s” can. This table is the root of the traceability thread from Section 2.1.

3.4 Architecture

Context diagram (C4 Level 1)

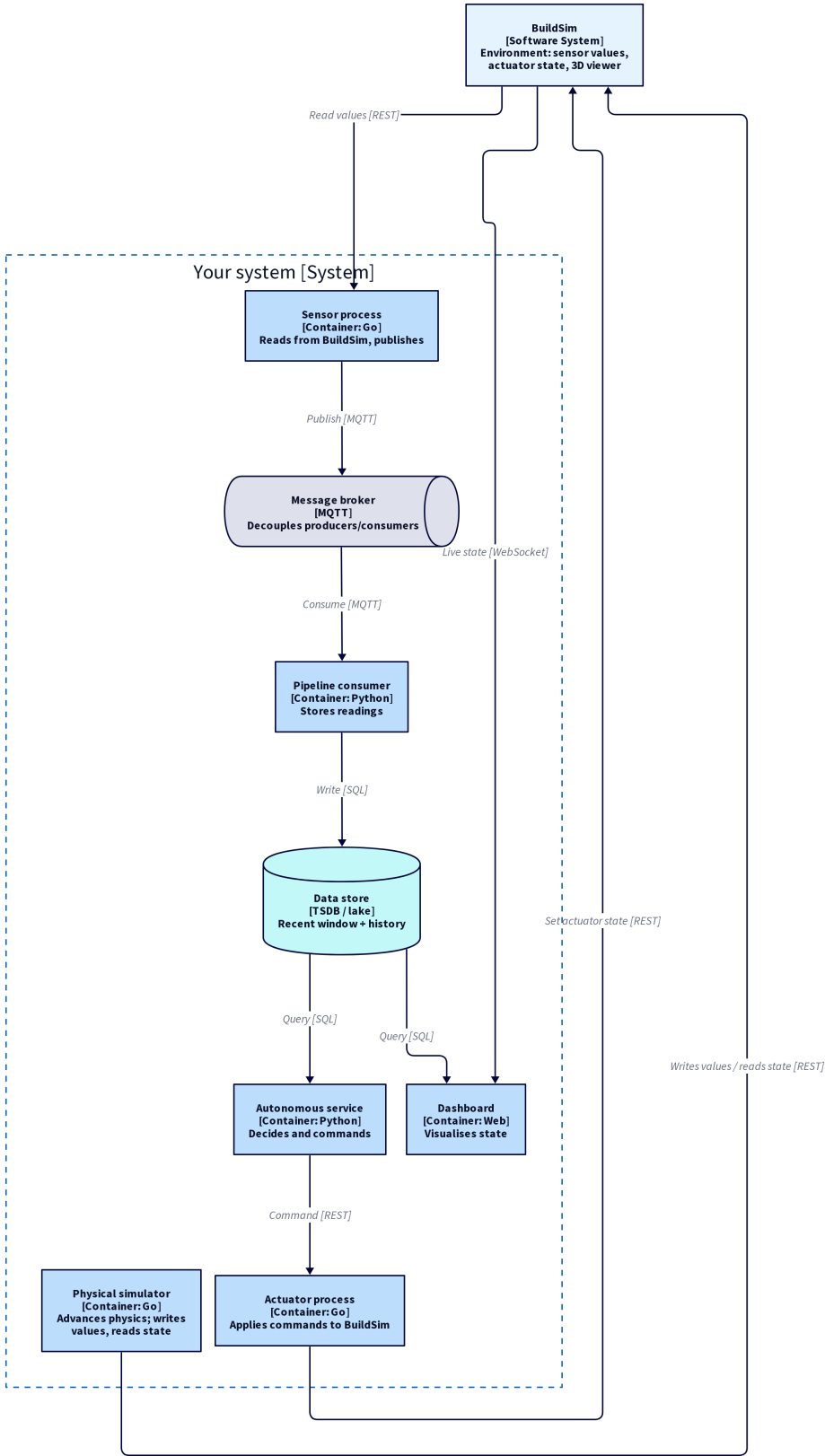
Show the whole system as one box, surrounded by the people and external systems it touches. Nothing internal. Describe each external actor in a sentence.



C4 System Context (Level 1). Person and software-system elements; the system in scope is highlighted, external systems are grey; every arrow is a directed relationship labelled with its purpose and protocol.

Container diagram (C4 Level 2)

Open the box: every deployable unit ($\approx$ one Docker container) and how they communicate, with the protocol on each arrow. This should map 1:1 to the `docker-compose.yml`.



C4 Container diagram (Level 2). Each container shows its [technology] and responsibility; the in-scope system is the dashed boundary, external systems (BuildSim) are grey; every arrow is a directed relationship labelled with its protocol.

Component responsibilities and design decisions

Every process should have a single primary responsibility. List every container with the **single job** it owns. For each component, explain not only *what* it does but **why it exists as a separate process** rather than being merged into another component, the “one job per process” principle is what makes the system testable and resilient, and it is exactly what the oral examination probes. Then give the **design-decisions table**: for each significant choice, the option chosen, the alternative(s) considered, the requirement it serves, and the trade-off accepted (the pattern from Section 2.6). This table carries most of the architecture grade.

3.5 Interfaces and communication

Specify each interface precisely enough that **another team could implement the system from this section alone**. State which communication pattern (Section 2.4) is used on each link and why. Give the **message schema** for every boundary crossing, field names, types, and units, and list each endpoint or topic with its payload and semantics (status codes, QoS, rate). Ambiguity here is the most expensive kind of bug: two components that look connected but disagree on the message format will work in isolation and fail when they meet.

3.6 Simulating the sensor values

Because BuildSim provides the rooms but not their physics, **the simulator that generates realistic sensor values must be built**. Be explicit about two things:

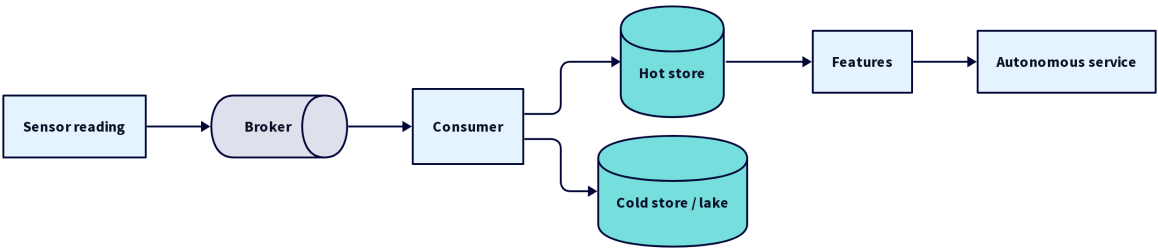
- **The physical processes.** How each sensor value is produced, for example a heat-balance update for temperature (envelope loss, body heat, HVAC), and a mass-balance for CO₂ that rises with occupancy and falls with ventilation. Crucially, **actuator commands must feed back into the model** so the control loop closes: raising the fan lowers CO₂, which the next reading reflects.
- **The movement of people (occupancy).** Occupancy usually drives everything else, heat, CO₂, lighting, so model it explicitly: a weekday schedule (arrivals 07:00–09:00, a lunch dip, departures 16:00–18:00), meeting-room bookings, empty weekends, and some randomness so no two days are alike.

Approaches and examples. Use a physics-based model (the equations above), data replay (feed a public dataset through the sensors, for example the UCI *Occupancy Detection* set or *ASHRAE* building-energy data), or a hybrid (real data with an injected event). The model need not be physically perfect; it must have enough structure, trends, daily cycles, genuine responses to actuators, that the autonomous service reacts to meaningful data. Be honest about its limitations.

3.7 Autonomous services and data engineering

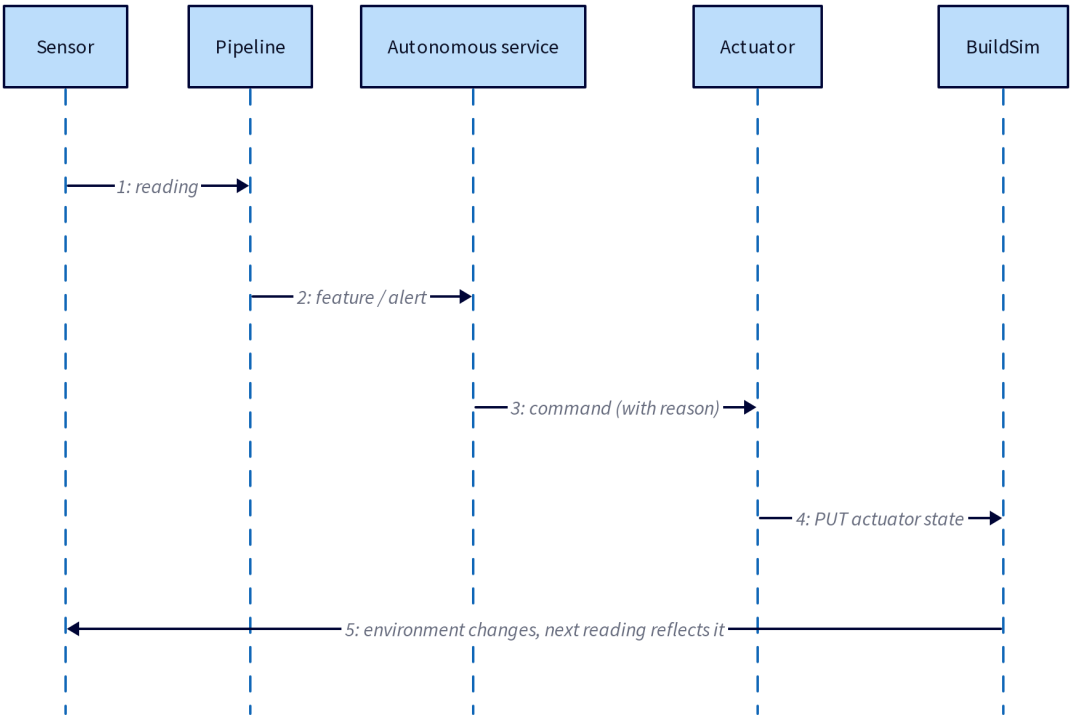
Describe the **autonomous service**, the component that turns data into action. Any approach is acceptable (rule-based, optimisation, ML, LLM, or a mix); choose the simplest that meets the requirement and justify it. **A more sophisticated AI technique does not automatically result in a higher grade.** Cover the approach and why it fits the latency budget, the inputs it uses, the action it produces, how it is integrated, how it is evaluated, and its failure/safety behaviour (what happens when it is wrong, slow, or unavailable, and the fallback that keeps the building safe).

Then describe the **data pipeline**: ingestion (is the broker a buffer?), storage (a time-series DB for the hot path, a data lake for history, and why), processing and feature engineering, and how **data quality** is handled (missing, stale, or duplicate readings). A good decision on bad data is still a bad outcome, so data quality is graded as seriously as the decision logic.

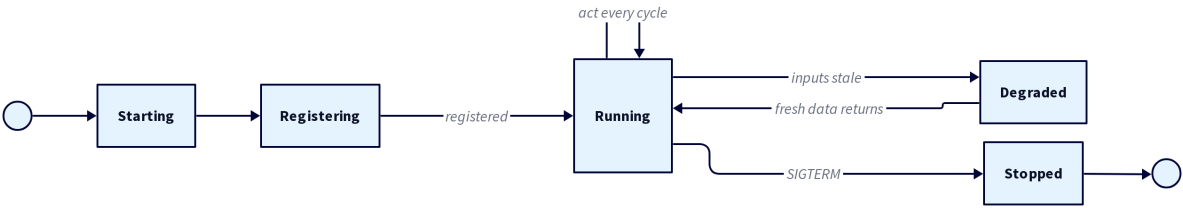


3.8 Behaviour

Static diagrams show structure; these show **behaviour over time**, which is where timing and failure bugs hide. Provide a **dynamic diagram** (C4’s name for a sequence-style view of one scenario, with numbered interactions among the containers) and a **state machine** for at least one process whose lifecycle matters (a sensor or the autonomous service), including what happens when the network drops and how it recovers. Every transition in the state machine is a piece of code that must be written.

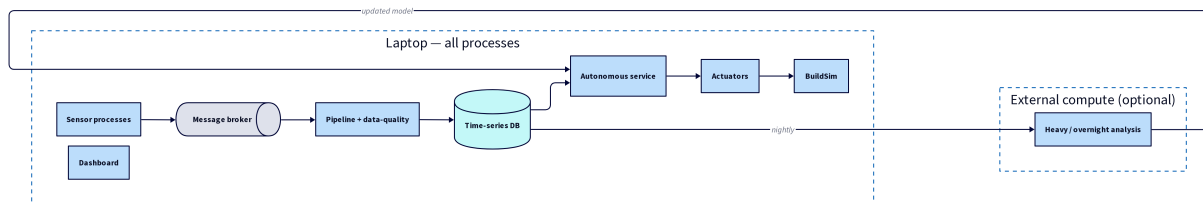


C4 Dynamic diagram, numbered runtime interactions among the containers for one scenario.



3.9 Deployment and component placement

Show **where each component runs and justify the placement**. Draw a deployment diagram with each process and where it runs (in this course, the laptop), then discuss **latency, fault isolation, resource requirements, maintainability, and operational considerations** (Section 2.5). Everything here runs on one machine, so most of this is about how the processes are composed, isolated, and communicate; if external compute such as a GPU server is used, explain why it is needed and how it affects the architecture. Explain **why each component is deployed separately and what would be lost if components were merged**. This is the view previous reports most often forget.



3.10 Test plan

Explain **how the system is tested and how evidence is collected**, not every individual test case. Every requirement in Section 3.3 must have a verification approach and a piece of evidence (a log, a measurement, a screenshot). Include at least unit tests, one end-to-end scenario test, and, for the higher grades, fault-injection tests (kill a process, send corrupt data) and quantitative measurements. This table is the validation end of the traceability thread.

3.11 Evaluation and experiments

Present the **evidence** about how well the system works and where it breaks. Report the metrics measured (latency, recovery time, false-alarm rate, energy). For the higher grades, include **performance plots and stress experiments**, a response-over-time plot, an energy comparison against a baseline, a scaling experiment that finds the breaking point, plus an analysis of the **trade-offs** observed and an **honest critique** of the weaknesses. “It works” is a grade-3 statement; “here is where it fails, why, and what would be changed” is a grade-5 statement. The worked example shows three such plots produced with `pgfplots`.

3.12 Dashboard, risks and reflection

Describe what the **dashboard** visualises (sensor readings, actuator states, decisions, alerts) and how it is built (BuildSim’s session API or a separate dashboard). Close with **risks, assumptions, and a critical reflection**: the things that could undermine the system, the assumptions the design depends on, and an honest account of its limitations and what would be done next. Being explicit here is a sign of engineering maturity, not weakness.

4. Checklist before submission

- ☐ Every requirement has an ID, an acceptance criterion, and a test (traceability holds).
- ☐ C4 context **and** container diagrams are present and consistent with the `docker-compose.yml`.
- ☐ Every significant design choice is justified against a named alternative, with a trade-off.
- ☐ Interfaces are specified precisely enough to implement from.
- ☐ The autonomous service and the data pipeline are both described and justified.
- ☐ At least one sequence diagram and one state machine are included.
- ☐ The deployment diagram shows where each component runs and justifies the placement.

- ☐ The evaluation reports measured evidence, finds a breaking point, and critiques the design honestly.
- ☐ Both team members can explain the whole system, the oral examination is individual.